Develop a GAN-based image augmentation system for enhancing medical images

To develop a GAN-based image augmentation system that generates additional synthetic grayscale medical-style images using a generator and discriminator network.

Algorithm
1. Import TensorFlow, NumPy and Matplotlib.
2. Create a small synthetic grayscale medical-style image dataset directly in Python.
3. Normalize the image pixel values to the range -1 to 1.
4. Build a generator network that accepts random noise and generates grayscale images.
5. Build a discriminator network to distinguish real and generated images.
6. Define binary cross-entropy loss for the two networks.
7. Initialize separate Adam optimizers for the generator and discriminator.
8. Train the discriminator using real and generated images.
9. Train the generator to produce images that are classified as real.
10. Repeat the adversarial training process for multiple epochs.
11. Generate additional grayscale images using random noise vectors.
12. Display the generated images as an image augmentation demonstration.

Program

```
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt
from tensorflow.keras import layers, models
tf.keras.utils.set_random_seed(38)
# Generate synthetic grayscale medical-style images
def make_scan(seed):
rng = np.random.default_rng(seed)
img = rng.normal(
0.0, 0.08,
(28, 28)
).astype("float32")
yy, xx = np.mgrid[-1:1:28j, -1:1:28j]

radius = np.sqrt(xx**2 + yy**2)
img += 0.45 * np.exp(
-((radius - 0.55)**2) / 0.025
)
img += 0.25 * np.exp(
-((xx + 0.25)**2 + (yy - 0.1)**2) / 0.08
)
img += 0.20 * np.exp(
-((xx - 0.25)**2 + (yy + 0.1)**2) / 0.08
)
return np.clip(img, 0, 1)
# Create dataset
images = np.array([
make_scan(i) for i in range(1000)
], dtype="float32")
images = images[..., None]
# Normalize
images = images * 2.0 - 1.0
noise_dim = 64
# Generator
generator = models.Sequential([
layers.Input(shape=(noise_dim,)),
layers.Dense(7 * 7 * 128, use_bias=False),
layers.BatchNormalization(),
layers.LeakyReLU(0.2),
layers.Reshape((7, 7, 128)),
layers.Conv2DTranspose(
64, 4,
strides=2,
padding='same',
use_bias=False
),
layers.BatchNormalization(),
layers.LeakyReLU(0.2),
layers.Conv2DTranspose(
```

```
1, 4,

strides=2,
padding='same',
activation='tanh'
)
])
# Discriminator
discriminator = models.Sequential([
layers.Input(shape=(28, 28, 1)),
layers.Conv2D(64, 4, strides=2, padding='same'),
layers.LeakyReLU(0.2),
layers.Dropout(0.3),
layers.Conv2D(128, 4, strides=2, padding='same'),
layers.LeakyReLU(0.2),
layers.Flatten(),
layers.Dense(1, activation='sigmoid')
])
# Loss and optimizers
bce = tf.keras.losses.BinaryCrossentropy()
g_opt = tf.keras.optimizers.Adam(1e-4)
d_opt = tf.keras.optimizers.Adam(1e-4)
# Training function
@tf.function
def train_step(real):
batch = tf.shape(real)[0]
noise = tf.random.normal(
[batch, noise_dim]
)
with tf.GradientTape() as dt, \
tf.GradientTape() as gt:
fake = generator(
noise,
training=True
)
real_score = discriminator(
real,
training=True
)

fake_score = discriminator(
fake,
training=True
)
d_loss = (
bce(tf.ones_like(real_score), real_score)
+
bce(tf.zeros_like(fake_score), fake_score)
)
g_loss = bce(
tf.ones_like(fake_score),
fake_score
)
dg = dt.gradient(
d_loss,
discriminator.trainable_variables
)
gg = gt.gradient(
g_loss,
generator.trainable_variables
)
d_opt.apply_gradients(
zip(dg, discriminator.trainable_variables)
)
g_opt.apply_gradients(
zip(gg, generator.trainable_variables)
)
return d_loss, g_loss
# Prepare dataset
dataset = (
tf.data.Dataset.from_tensor_slices(images)
.shuffle(1000)
.batch(64)
```

```
)
# Train GAN
for epoch in range(10):
d_losses = []
g_losses = []

for batch in dataset:
dl, gl = train_step(batch)
d_losses.append(float(dl))
g_losses.append(float(gl))
print(
f"Epoch {epoch + 1}: "
f"D Loss={np.mean(d_losses):.3f}, "
f"G Loss={np.mean(g_losses):.3f}"
)
# Generate images
samples = generator(
tf.random.normal([9, noise_dim]),
training=False
).numpy()
samples = (samples + 1) / 2
# Display images
fig, axes = plt.subplots(3, 3, figsize=(6, 6))
for ax, image in zip(axes.flat, samples):
ax.imshow(
image.squeeze(),
cmap='gray'
)
ax.axis('off')
plt.tight_layout()
plt.show()
```

Output of Program

```
Epoch 1: D Loss=... G Loss=...
Epoch 2: D Loss=... G Loss=...
Epoch 3: D Loss=... G Loss=...
...
Epoch 10: D Loss=... G Loss=...
```

Generated grayscale medical-style images
are displayed in a 3 × 3 grid.

Result

Thus, a GAN-based image augmentation system was implemented using synthetic grayscale medical-style images. The generator produced additional images using random noise. The dataset is simulated for laboratory demonstration and is not suitable for clinical diagnosis.